

Securing Autonomous UAV Cluster With Blockchain-Based Threshold Key Management System Utilizing Crypto-Asset and Multisignature

Mebanjop Kharjana^{1b}, Subhas Chandra Sahana^{2b}, and Goutam Saha^{1b}

Abstract—Unmanned aerial vehicles deployed in remote locations rely on self-governed key management for their protection. However, conventional key management depends on a centralized ground-based station or single vehicle. Such a system is vulnerable to compromised certificate authority problems and single-points-of-failure. This paper proposed to resolve these vulnerabilities using a blockchain-based threshold key management system. The proposed system utilized blockchain's concepts of crypto-asset and multisignature. Keys are defined as crypto-assets to improve their management in the blockchain network. Multisignature facilitates collaboration during key management based on a threshold value. The threshold value is also configurable to meet systems' security and performance requirements. The proposed system secured the process of re-enforcement, sub-clustering, re-merging, and inter-cluster migration. Security analysis revealed that the proposed system complied with most key management security guidelines. The custom signature module used to authenticate intra-cluster communication was also verified as safe. Threats to the cluster were identified, assessed for risk, and mitigated accordingly. Performance analysis found that both AODV and DSDV routing protocols offer consistent performance but DSDV prevailed during the worst-case network scenario. The paper finally identified research gaps, including the requirement for an optimized mechanism for collecting consent signatures.

Index Terms—Autonomous UAV cluster, threshold key management system, blockchain, crypto-asset, multisignature.

I. INTRODUCTION

UNMANNED aerial vehicle (UAV) is used primarily for military applications like real-time surveillance, intelligence data gathering, high-risk missions, and aerial warfare. It is now used in other broad spectra of civilian applications such as search and rescue operations, wildlife tracking, weather monitoring, pollution control, etc. It is also an innovative technology deployed in clusters to transform the operations of different businesses and industries. Multiple UAVs are inter-connected to form a cluster and cooperate to achieve a common goal. Such a cluster is configured as an underlying network to support various

commercial applications, from mapping software to logistics operations. However, different types of attacks [1], [2], [3] on UAVs can affect their hardware, software, and networking operations. Such attacks can significantly disrupt the normal functioning of the UAV cluster and the business operations that rely on it. Therefore, securing the UAV cluster is very important, especially when deployed at remote locations for critical military or civil operations.

The important security aspect of a UAV cluster when operating at a remote location is key management. Key management is used by UAVs to perform (i) identification, (ii) authentication, and (ii) secured communication. The common approach is to designate a single UAV as a centralized certification authority (CA) to manage key information. However, such a CA can be compromised affecting normal key management services. To address this issue, different collaborative key management systems are proposed. Key management systems [4], [5], [6] propose a threshold mechanism based on Shamir's Secret Sharing (SSS) to secure wireless sensors and other ad-hoc networks. The SSS shares a master key pair across the network with a (k, n) threshold. At least k of the total n shareholders must collaborate to perform key generation service. The threshold technique is also adopted to manage network group keys and prevent attacks from malicious actors. Further, a group key agreement protocol [7] integrates SSS with a modified centralized circular hierarchical (CCH) group model to collect m shares from n mobile nodes to generate and distribute a group key. However, SSS and its extension depend on a centralized party to construct a final key from the collected secrets. If such a party fails or is compromised, it can cause a single-point-of-failure event and even jeopardize the whole system. The current trend to address the issues of compromised CA and single-point-of-failure is to use blockchain. Blockchain is a distributed ledger technology used for secured record saving and tracking. Data on the blockchain are transparent and immutable, making it suitable for managing key information. A blockchain ledger is also distributed across the network, enhancing its resilience and resistance to network failures. Distributed authentication services [8], [9] utilize blockchain-based mechanisms to secure UAV networks. They use a blockchain ledger as a distributed and immutable authentication information storage. They also deploy smart contracts as a means to provide authentication services. Shamir's Secret Sharing is also used in blockchain-based security systems [10], [11] to facilitate threshold key-sharing

Received 7 July 2024; revised 17 November 2024; accepted 27 January 2025. Date of publication 4 February 2025; date of current version 5 June 2025. Recommended for acceptance by X. Yuan. (Corresponding author: Mebanjop Kharjana.)

Mebanjop Kharjana and Goutam Saha are with the Department of Information Technology, North-Eastern Hill University, Shillong 793022, India (e-mail: meban@nehu.ac.in; gsaha@nehu.ac.in).

Subhas Chandra Sahana is with the Department of Computer Science and Engineering, National Institute of Technology, Durgapur 713209, India (e-mail: scsahana.cse@nitdgp.ac.in).

Digital Object Identifier 10.1109/TMC.2025.3538462

mechanism. The private key is split into multiple sub-keys based on Shamir's threshold scheme to prevent any loss and disclosure of the original key. These systems also support authentication and encryption techniques to secure wireless communications.

Recent developments of blockchain's crypto-assets [12] and tokens [13] introduce new research opportunities. Non-fungible tokens (NFTs) are used by authentication systems [14], [15], [16] to identify users and devices. Token-based identities are unique and immutable, while the underlying blockchain ensures they are shareable and verifiable. NFTs and smart contracts are leveraged by [17] to secure its Public Key Infrastructure (PKI) Ecosystem. While NFTs represent digital assets, smart contracts provide for their execution logic. NFTs' ownership is established through a digital signature, which is utilized for the authentication process. Similarly, decentralized identities are constructed for digital objects by an identifier management scheme [18]. They are NFT-based identifiers that are managed on the blockchain network using smart contracts. NFT ensures that ownership of digital objects is unique and authentic. Blockchain-based token technology is also used to support Self-Sovereign Identity (SSI) [19] in Metaverse. NFTs are held by users as their unique identifier and manage them independently. NFT with Zero Proof knowledge is used as evidence for trust in the Metaverse. The NFT-based identifier is also used for authentication purposes to access services.

Following the current trend, this paper aims to secure autonomous UAV clusters using a blockchain-based threshold key management system. It fully utilizes advanced blockchain features to (i) further enhance existing key management systems, (ii) address persisting security issues relating to key management, and (iii) design a configurable system that meets different security and performance requirements of UAV applications.

II. RELATED WORKS

A secure blockchain-based key management scheme [20] was proposed for an autonomous flying ad hoc network (FANET). A head UAV was used to provide key management services and has higher storage and computation capability. Key management was also carried out autonomously without the intervention of the ground station. This was possible as blockchain was used as the trusted key store. The proposed scheme also supported key revocation and UAV migration. The scheme had minimal energy requirements and was proven secure against different security attacks. An authentication model [21] was proposed that leveraged decentralized blockchain-based security. It addressed the issue of low latency during drone authentication in smart city applications. It implemented a zone-based architecture, with each zone managed by a drone controller. The drone controller handled authentication, migration, and secure communication. The drone controller maintained a blockchain ledger and used a customized consensus algorithm of drone-based delegated proof of stake (DDPOS). This proposed model offered a low packet loss rate, high throughput, and low end-to-end delay. A blockchain-based public key infrastructure (PKI) [22] was proposed to perform decentralized authentication for UAV networks. The blockchain stored public keys, identities, and trust

relationships as global trust graphs. UAV only stored a part of the blockchain that is relevant to it. Two UAVs combine their knowledge to authenticate and find a trustworthy path in their trust graph. The blockchain also acted as a secured storage and access controller for the trust graph in the PKI. A blockchain-based cross-domain authentication service [23] was proposed for intelligent 5G-enabled internet of drones. It addressed the issue of single-point-of-failure and cross-domain authentication in drone networks. It also supported federated identity for collaborative domains using a multi-party signatures scheme. It facilitated the process of drone joining and exiting between different domains. A smart contract performed cross-domain authentication and established session keys to ensure reliable communication. The system was proven efficient and resistant to security threats. An identity and aggregate signature-based authentication protocol [24] was proposed for the internet of drones. The network consisted of two types of drones, reconnaissance and attacking. These drones communicated with the remote controller through a wireless network or satellite using GPS (Global Positioning System) signals. Two protocols were designed and used to guarantee data integrity, authorization, and confidentiality of communications. Pairing cryptography generated the public-private key pairs for drone identification. The authentication scheme used aggregate signatures to ensure protection against GPS spoofing attacks. Key exchange was performed using the Computational Diffie-Hellman Problem (CDHP) [25]. The proposed protocol was verified using a random oracle model (ROM), a real-or-random (ROR) model, and mathematical lemmas.

Alternate approaches were identified in which customized blockchain architectures and features were employed to secure UAV networks. A multi-layered blockchain architecture [26] was designed to manage group keys for an urban UAV network. This hierarchical blockchain structure assisted in determining node mobility and group density. Custom blockchain block and transaction types were used by a task management scheme [27] to maintain identity information and secure task management in a multi-UAV network. It also utilized a special consensus mechanism to accelerate the consensus between the server network and the ground station. NFT was used in DroneXNFT [28] to manage flight data and secure autonomous UAV operations. Further, advanced technologies like deep learning were also integrated with blockchain to devise advanced security frameworks. An intelligent fuzzy blockchain framework [29] utilized novel fuzzy deep learning algorithms to detect network attacks in blockchain-based IoT environments.

III. MOTIVATION

Investigations revealed that most key management services are centralized which leads to single-point-of-failure. These systems are vulnerable if the centralized authority is compromised. It was noticed that most systems use blockchain only as a storage and distributing medium. The blockchain remained under-utilized with its advanced features not leveraged to address pending key management issues. Moreover, most key management systems depend on fixed infrastructure and are

unsuitable when a UAV cluster is deployed to remote locations. These systems are also not configurable to meet the varying security and performance requirements of UAV applications. Based on these observations, research initiatives had been undertaken that focused on the following:

- To design a blockchain-based threshold key management system to secure an autonomous UAV cluster by utilizing advanced features of blockchain.
- To distribute key management authority among multiple entities to resolve the compromised CA problem and the single-point-of-failure event.
- To make key management system configurable to meet varying security and performance requirements.
- To ensure the key management system complies with established security guidelines.

IV. ORGANIZATION

The rest of the paper is organized as follows. Section V presents an overview of the proposed key management system. Section VI implements the proposed key management system for autonomous UAV clusters. Section VII and Section VIII identify methodologies relating to the security and performance analysis respectively. Section IX discusses results data relating to the security and performance analysis. Section X identifies the future scope of work to improve the proposed key management system. Finally, the paper concludes in Section XI.

V. METHODOLOGY

This paper proposes a blockchain-based threshold key management system for an autonomous UAV cluster. The proposed system leverages two features of blockchain technology, namely (i) *crypto-asset* and (ii) *multisignature* (or multisig). The core idea of the proposal is to treat a key as a crypto-asset. A crypto-asset is bound to a key and assigned a unique asset name at creation time. This asset name identifies the key during different key management operations. A blockchain network is used as a medium to manage and secure keys as crypto-assets. The *m-of-n* multisignature is then used to entitle ownership and usage rights of the crypto-asset to multiple owners. For this purpose, a *redeem script* based on Bitcoin [30] is calculated using a threshold value m and the public keys of the n owners. A *locking script* (also referred to as multisignature) is then constructed from the *redeem script*. Such *m-of-n* multisignature implies that the crypto-asset is collectively owned by n owners but consent signatures from at least m owner(s) are required to modify such assets. These m signatures are presented as *unlocking script* to prove asset ownership and authorized asset modification. The parameter m serves as a (i) *functional threshold* to facilitate collaborative operations, and (ii) *security threshold* beyond which malicious actors can collaborate to compromise the system. As shown in Fig. 1, an asset-based certification of a public key is proposed that uses asset name for identification and multisignature for multi-party certification.

The proposed system aims to be configurable to meet different security and performance requirements in a UAV cluster. For this purpose, a $M:N:T$ cluster configuration is defined as shown

Standard Public Key Certificate		Asset-based Public Key Certificate	
Serial No.		Asset Name	
Key Information		Key Information	
CA Information		Multisignature (Locking Script)	
CA Signature			
Redeem Script	m PubKey-1 ... PubKey-n n OP_CHECKMULTISIG		
Locking Script	OP_HASH160 <20-byte hash of redeem_script> OP_EQUAL		
Unlocking Script	Sig-1 ... Sig-m redeem_script		

Fig. 1. Standard public key vs. Asset-based public key certifications.

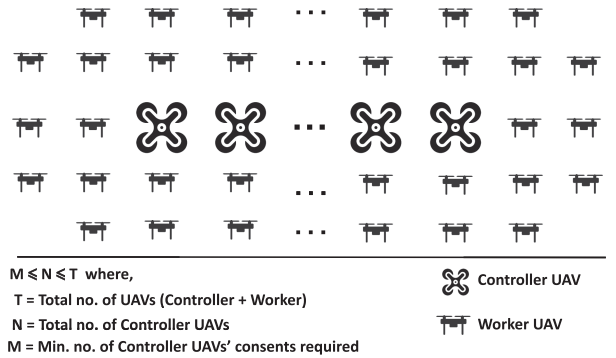


Fig. 2. Cluster with $M:N:T$ configuration.

in Fig. 2. It consists of T number of UAVs. Out of T UAVs, N is identified as Controller UAV (CUAV), and the remaining $(T-N)$ as Worker UAV (WUAV). WUAVs are members entrusted to perform the actual work in the cluster. They have limited computational and storage resources. CUAVs are administrators that control key management operations in the cluster. They have higher computational and storage capabilities. CUAVs hold a copy of the blockchain and maintain consensus with each other. The threshold value M defines the minimum number of CUAVs required to perform asset/key management operations. Moreover, CUAVs are responsible for managing cluster keys and ensuring the confidentiality of cluster communication. Hence, CUAVs form the backbone of key management operations for an autonomous UAV cluster without relying on fixed infrastructure.

For a cluster with $M:N:T$ configuration, the parameter values are selected based on the following criteria:

- $M \leq N \leq T$.
- $M > \frac{1}{2}N$ (to prevent a minority group of compromised CUAVs from running malicious parallel operations).

The parameter M directly affects the performance and security of the cluster. Consider cluster configuration where (i) $M=1$, $N=T$: All members are of the CUAV type. It offers high performance as only one consent is required, but a single CUAV can compromise the whole cluster, (ii) $M=N=1$, $T=*$: It is equivalent to a centralized system with a single CUAV controlling the cluster. It offers high performance but suffers from lower

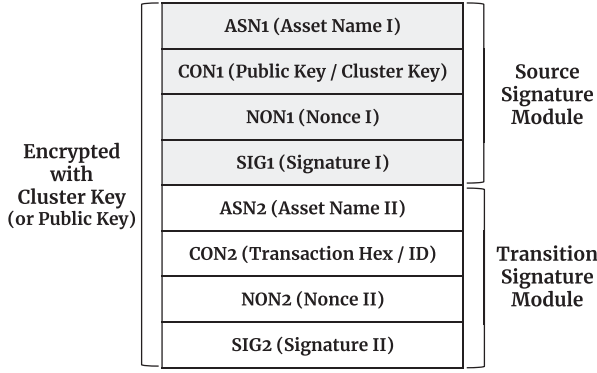


Fig. 3. Proposed signature module.

TABLE I
CRYPTOGRAPHIC NOTATIONS

	Description
$hsh\{A\}$	Hash value of a content A.
$enc\{A\}_{KY}$	Encrypt content A with a key KY.
$sig\{A\}_{SY}$	Sign content A with private key SY.
$ver\{A, B\}_{PY}$	Verify signature A with B using public key PY.
TX^k	Multisig transaction that received k signatures.
η	Nonce value used to prevent replay attacks. e.g. random no., timestamp, etc.

security, (iii) $M=N$, $T=*$: It suffers from low performance as the consent of all CUAVs is required. It will be non-operational even if one CUAV is compromised, (iv) $M=N=T$: It consists of only CUAVs and performs poorly as all their consent is required. It will be non-operational even if just one member is compromised.

Therefore, parameter values for cluster configuration $M:N:T$ depend on the operation requirement. For performance, the value of M should be lower so that only fewer consents are required to complete a key management operation. For better security, the value of M should be higher so that if a CUAV is compromised, it cannot singly manipulate key information. Moreover, a higher N value ensures that the system can fall back on the support of other CAUVs if a single CUAV is destroyed.

Moreover, the proposed key management system involves intra-cluster communication among UAVs. A custom signature module is proposed to authenticate such communication. As shown in Fig. 3, the custom signature module is divided into the (i) Source Signature Module (SSM) and (ii) Transition Signature Module (TSM). The SSM stores the authentication data of the requester, whereas the TSM stores the authentication data of the latest intermediate participant. In case of a key update operation, $ASN1$ stores the asset name of the key to be updated. The $CON1$ field stores the newly generated public key, and $SIG1$ is a signature calculated from $ASN1$, $CON1$, and $NON1$. Similarly, $ASN2$ stores the asset name of the key that had last consented to the update request. The $CON2$ contains the blockchain transaction in hex format, and signature $SIG2$ is calculated from $ASN2$, $CON2$, and $NON2$. $NON1$ and $NON2$ are randomly generated nonce values for preventing replay attacks. Authentication using SSM and TSM is performed as shown in Algorithm 1. Note that the authenticating key is not transmitted with the module but retrieved using its asset name.

Algorithm 1. Verify Signature Modules of Payload

```

VALIDATE (Payload)
  Compute  $HS1 \leftarrow hsh\{AS1, CON1, NON1\}$ 
  Compute  $HS2 \leftarrow hsh\{AS2, CON2, NON2\}$ 
  If  $(ver\{SIG1, HS1\}_{Public\ Key(AS1)})$  and
 $(ver\{SIG2, HS2\}_{Public\ Key(AS2)})$  then
    return True
  else
    return False

```

VI. IMPLEMENTATION OF BLOCKCHAIN-BASED THRESHOLD KEY MANAGEMENT SYSTEM UTILIZING CRYPTO-ASSET AND MULTISIGNATURE

Different cryptographic functions used are illustrated in Table I. The flow of communication from A to B is represented using the notation of $A \rightarrow B : \{Payload\}$. The implementation of the blockchain platform is based on the premise of MultiChain. MultiChain is an off-the-shelf private blockchain platform whose application programming interface (APIs) [31] are compatible with those of Bitcoin Core [32]. The proposed system utilizes API commands that include *createmultisig*, *grant*, *issue*, *getassetinfo*, *createrawsendfrom*, *signrawtransaction* and *sendrawtransaction*. The proposed system can be adapted to other blockchain platforms that support crypto-assets and multisignature (or smart contract) functionalities. Most modern blockchains support these features and the proposed system is deployable on such platforms.

A. Configure UAV Cluster

For a cluster with $M:N:T$ configuration, CUAV[i] and WAUV[i] represent the i^{th} Controller/Worker UAV, respectively. The ground control station (GCS) pre-configures CUAVs and WUAVs to perform subsequent key management operations autonomously.

1) *Setting up Cryptographic Parameters*: GCS uses elliptic curve cryptography (ECC) as the public key algorithm. It also uses the Elliptic Curve Digital Signature Algorithm (ECDSA) as a digital signature scheme. GCS establishes initial agreement on the elliptic curve E with a generator point g of the order n . Using the curve parameters (E, g, n) , GCS generates a private-public key pair as its identity. If GSK is the private key, its corresponding public key GPK is calculated as $GPK = g * GSK$. The curve parameters (E, g, n) are then published to both CUAVs and WUAVs of the cluster.

2) *Generating Cryptographic Identities for UAVs*: CUAV/WUAV generates private-public key pairs as its identity using the curve parameters (E, g, n) . Individual UAV secures their private key and does not depend on the GCS. For CUAV[i]/WUAV[i], the private-public key pair is denoted as $(CSK[i], CPK[i])$ / $(WSK[i], WPK[i])$. All UAVs then send their newly generated public keys to the GCS for identification and registration.

3) *Setting up Cryptographic Identity for Cluster*: A cryptographic identifier is assigned to the UAV cluster by deriving it from the identities of CUAVs. For this purpose, GCS calculates

the cluster identifier MSA from the public keys of all CUAVs, as shown in (1).

$$MSA = createmultisig$$

$$M \{ CPK[0], CPK[1], \dots, CPK[N-1] \} \quad (1)$$

Note that the cluster identifier MSA is simply a multisig address. It enforces that of all N CUAVs, at least M must consent to complete a key management operation. In addition, the (2) is executed to permit the cluster MSA to create new assets during the re-enforcement process (see Section VI-E5).

$$grant \ MSA \ issue \quad (2)$$

4) *Registering Cryptographic Identity of GCS and UAVs:* GCS registers public keys of WUAVs/CUAVs as blockchain-based crypto-assets. The crypto-asset essentially consists of (i) unique asset name, (ii) key information, and (iii) ownership information. The asset name is used to identify and access the crypto-asset and its associated key. Key information stored in the crypto-asset is the public key of a UAV. The ownership information contains ownership data of the crypto-asset. For WUAV[i], a crypto-asset named WAST#i is created and associated with its public key WPK[i]. The ownership of the crypto-asset WAST#i is set to MSA. GCS carries out the registration process, as shown in (3).

$$\begin{aligned} &issue \ MSA \ \{ \ name : \ WAST\#i \} \\ &\{ \ pubkey : \ WPK[i] \} \end{aligned} \quad (3)$$

MSA owns the crypto-asset WAST#i and its public key WPK[i], so only CUAVs can manage it. In addition, the public keys of CUAVs are also registered as crypto-assets. However, their public keys are used to calculate MSA and should therefore be immutable. For this purpose, ownership of their crypto-assets is set to a *burn address*. A burn address is an address whose corresponding private key is not retrievable. A crypto-asset owned by a burn address is immutable and nontransferable. If BAD is a burn address, the crypto-asset for a CUAV[i] named CAST#i is created whose ownership is set to BAD using the same step as in (3). Similarly, the identity of GCS is also registered as an immutable crypto-asset named GAST using (3). Registered crypto-asset and its associated key information can be retrieved from the blockchain network. To retrieve the crypto-asset of WUAV[p], it is done by specifying its crypto-asset name of WAST#p, as shown in (4).

$$getassetinfo \ WAST\#p \quad (4)$$

GCS then creates two lists: a list of Controller UAVs (LCU) and a list of Worker UAVs (LWU). LCU is immutable and contains the public keys of CUAVs and their corresponding asset names. It is denoted as $LCU = \{(CAST\#0, CPK[0]), \dots, (CAST\#(N-1), CPK[N-1])\}$. In contrast, LWU is a dynamic list and contains the public keys of WUAVs and their corresponding asset names. It is denoted as $LWU = \{(WAST\#0, WPK[0]), \dots, (WAST\#(T-1), WPK[T-1])\}$. The LCU and LWU are sent to all the WUAVs and act as their trusted source. CUAVs may optionally save copies of the LCU and LWU to facilitate quick

lookup. Each UAV also saves key information related to the crypto-asset GAST of the GCS.

5) *Setting up the Blockchain Network:* GCS loads each CUAV with a copy of the blockchain ledger that contains crypto-assets information. CUAVs are set up as full nodes and can validate underlying transactions and blocks. They are configured with individual wallets and authorized to call different blockchain-based APIs. CUAVs form the backbone of the blockchain network to synchronize key information.

Performance and scalability are major hurdles in the practical application of blockchain. Most traditional blockchains have low transaction throughput, high transaction confirmation latency, and excessive energy consumption. As a result, many scalability issues arise when the number of users increases. MultiChain addresses these issues by utilizing a delegated consensus mechanism instead of the proof of work (POW). A round-robin technique is used to select a miner for a particular round. For this purpose, a *mining diversity* parameter is defined whose value is $0 \leq mining \ diversity \leq 1$. A *spacing* value is then calculated as the round-up product of the total number of miners and the mining diversity. A new block will be accepted only if its miner did not mine any previous *spacing*-1 blocks. This round-robin technique addresses two main issues: (i) mining freeze due to inactive miners and (ii) malicious miners building an alternative blockchain. A selected miner performs mining by signing the new block to prove its identity. As a result, this mining technique of MultiChain can confirm 2,000 transactions/sec (2,500 transactions/sec without confirmation) and allows the blockchain network to scale.

B. Cluster Key Management

Intra-cluster communication is secured via symmetric encryption using a shared cluster key. GCS generates the initial cluster key CK_o and shares it with all UAVs. To improve security, the cluster key is updated at regular intervals. For this purpose, the system randomly selects a CUAV to generate subsequent cluster keys at regular intervals. Of the N CUAVs, one CUAV is chosen using the round-robin technique mentioned in Section VI-A5. Scheduling cluster key generation based on such a technique ensures each CUAV equal opportunity and prevents inactive CUAVs from disrupting the process. If CUAV[s] is selected, it generates a new cluster key CK_1 and broadcasts it to other UAVs. Broadcasting CK_1 to CUAV[x]/WUAV[x] is done as shown in (5) and (6).

$$\begin{aligned} &CUAV[s] \rightarrow CUAV[x] : \{ enc\{ CAST\#s, \\ &CK_1, \eta_1, sig\{ hsh\{ CAST\#s, CK_1, \eta_1 \} \} \\ &CSK[s], -, -, -, - \} CK_o \} \end{aligned} \quad (5)$$

$$\begin{aligned} &CUAV[s] \rightarrow WUAV[x] : \{ enc\{ CAST\#s, \\ &CK_1, \eta_2, sig\{ hsh\{ CAST\#s, CK_1, \eta_2 \} \} \\ &CSK[s], -, -, -, - \} CK_o \} \end{aligned} \quad (6)$$

Note that the symbol (-) is used wherever data is not required. The payload is per the format given in Fig. 3 and encrypted using the current cluster key CK_o . The new cluster key CK_1

is accepted only if SSM is authenticated. In addition, using the round-robin scheduling technique of MultiChain, CUAVs can further verify CUAV[s] as the authorized cluster key generator. However, WUAVs cannot access the blockchain and its scheduling technique. An alternate solution is for other CUAVs to rebroadcast CK₁ to WUAVs. WUAVs accept CK₁ only if they receive rebroadcasts from at least M CUAVs.

C. Key Update

WUAVs constantly update their key pair to prevent any potential key leakage. Consider that WUAV[j] wants to update its existing key pair of (WSK[j], WPK[j]). Using the curve parameters (E, g, n), WUAV[j] selects WSK'[j] as its new private key and calculates the corresponding public key as $WPK'[j] = g * WSK'[j]$. Then consent signatures should be collected from at least M CUAVs to update WPK[j] to WPK'[j]. Assuming that these M CUAVs are identified as {CUAV[0]...CUAV[M-1]}. As shown in (7), the first request for consent is sent to CUAV[0].

$$\begin{aligned} WUAV[j] \rightarrow CUAV[0] : \{ &enc\{ WAST\#j, \\ &WPK'[j], \eta_3, sig\{ hsh\{ WAST\#j, \\ &WPK'[j], \eta_3 \} \}_{WSK[j], WAST\#j, -, \eta_4, \\ &sig\{ hsh\{ WAST\#j, -, \eta_4 \} \}_{WSK[j]} \}_{CK_1} \} \end{aligned} \quad (7)$$

When CUAV[0] receives the request, it decrypts the payload using the cluster key CK₁ and validates it using SSM and TSM. CUAV[0] then gives its consent signature to update key information of the crypto-asset WAST-j to WPK'[j]. For this purpose, a multisig transaction is constructed and signed, as shown in (8).

$$\begin{aligned} TX^1 = createrawsendfrom MSA \\ \{ MSA : \{ WAST\#j : 1 \}, Data : \\ \{ pubkey : WPK'[j] \} \} (action = sign) \end{aligned} \quad (8)$$

The *action* parameter is set to *sign* so that CUAV[0] signs the resulting transaction TX^1 . TX^1 is a MultiChain object that consists of two parts: (i) a blockchain transaction in hexadecimal format. The notation 1 indicates that the transaction is signed only once, (ii) a *complete* field representing the status of the multi-signing process. The complete field value is set to *true* only when the transaction received M numbers of consent signatures else *false*. Therefore, TX^1 is forwarded to other CUAVs to collect their consent signatures. As shown in (9), CUAV[0] sends a new request with TX^1 as its content and forward it to CUAV[1].

$$\begin{aligned} CUAV[0] \rightarrow CUAV[1] : \{ &enc\{ WAST\#j, \\ &WPK'[j], \eta_3, sig\{ hsh\{ WAST\#j, \\ &WPK'[j], \eta_3 \} \}_{WSK[j], CAST\#0, TX^1, \eta_5, \\ &sig\{ hsh\{ CAST\#0, TX^1, \eta_5 \} \}_{CK_{[0]}} \}_{CK_1} \} \end{aligned} \quad (9)$$

Similarly, CUAV[1] receives the request and decrypts the payload using CK₁. If SSM and TSM are authenticated, then CUAV[1] consents and signs the transaction (Hex) part of TX^1

as shown in (10).

$$\begin{aligned} TX^2 = signrawtransaction \\ < Transaction (Hex) part of TX^1 > \end{aligned} \quad (10)$$

The object TX^2 contains a multisig transaction signed by two CUAVs, i.e., CUAV[0] and CUAV[1]. This process of signing and forwarding the partially signed multisig transaction continues until the request reaches the CUAV[M-1]. When CUAV[M-1] consented, it signs the transaction using the same command shown in (10). The output is TX^M , which contains a multisig transaction signed with M consent signatures, and its *complete* field set as *true*. As shown in (11), CUAV[M-1] then submits the transaction (hex) part of TX^M to the blockchain network.

$$\begin{aligned} TX_ID = sendrawtransaction \\ < Transaction (Hex) part of TX^M > \end{aligned} \quad (11)$$

CUAV[M-1] receives a TX_ID after submitting the transaction to the blockchain network. It then broadcasts the new key WPK'[j] to every WUAV in the cluster. Broadcasting of WPK'[j] to WUAV[x] is shown in (12).

$$\begin{aligned} CUAV[M-1] \rightarrow WUAV[x] : \{ &enc\{ WAST\#j, \\ &WPK'[j], \eta_3, sig\{ hsh\{ WAST\#j, WPK'[j], \\ &\eta_3 \} \}_{WSK[j], CAST\#(M-1), TX_ID, \eta_6, \\ &sig\{ hsh\{ CAST\#(M-1), TX_ID, \\ &\eta_6 \} \}_{CK_{[M-1]}} \}_{CK_1} \} \end{aligned} \quad (12)$$

When WUAVs receive the payload, they decrypt and validate it using SSM and TSM. WUAVs then update the crypto-asset WAST#j in their LWUs with a new key of WPK'[j]. Broadcasting to CUAVs is unnecessary as they can retrieve updated key information directly from the blockchain network.

D. Key Revocation

The proposed system allows the revocation of a crypto-asset associated with a given key. For this purpose, the ownership of a crypto-asset is changed from an MSA to a burn address. If REV is identified as a burn address, a crypto-asset is revoked by changing the ownership from MSA to REV. Consider that revocation of the asset WAST#k associated with WUAV[k] is required. Then any CUAV can initiate the revocation process, as shown in (13).

$$\begin{aligned} RX^1 = createrawsendfrom MSA \\ \{ REV : \{ WAST\#k : 1 \}, Data : \\ \{ pubkey : WPK[k] \} \} (action = sign) \end{aligned} \quad (13)$$

RX^1 object is then forwarded to other CUAVs to collect at least M consent signatures and revoke the crypto-asset WAST#k. This is similar to the key update process given in (9), (10), (11), and (12). Revocation of a crypto-asset is a permanent procedure and not reversible. However, note that the crypto-assets of CUAVs are immutable and not revocable.

E. Special Scenarios

1) *Permission Control on Crypto-Asset*: MultiChain allows management of the global permissions of the MSA identifier. Two commands are used, namely *grant* and *revoke*. The *grant* command is used to give permissions to MSA to perform *issue*, *create*, *send*, and *receive* on the crypto-asset. On the other hand, the *revoke* command nullifies permissions granted to MSA. In addition, MultiChain is configurable such that an administrator cannot singly grant or revoke permissions. Changes relating to grant/revoke are applied only if a certain number of administrators agree. This prevents a compromised administrator from jeopardizing the blockchain network.

2) *WUAV failure/compromise*: WUAV does not control key management operations. Hence, a compromised or non-operational WUAV has a limited effect on the cluster. Any CUAV can initiate the revocation process of such WUAV as mentioned in Section VI-D. Moreover, adding new WUAV to such a cluster is possible via the re-enforcement process discussed in Section VI-E5.

3) *CUAV failure/compromise*: Key management for a cluster with a $M:N:T$ configuration remains (i) *operational* as long as M or more CUAVs are active. It becomes non-operational if the number of active CUAVs becomes less than M , (ii) *secure* as long as the total number of compromised CUAVs does not exceed M . If it equals or exceeds M , then such CUAVs can collaborate to compromise the system. Note that adding a new CUAV to the cluster via the re-enforcement process (refer Section VI-E5) is not supported.

4) *Single-Point-of-Failure Issue*: The single-point-of-failure issue can be discussed based on the key management and the network. (i) Regarding key management, it is resolved by taking advantage of the multisignature scheme. For a given $M:N:T$ configuration, key management depends only on M numbers of CUAVs. The system remains operational even if the remaining $(N-M)$ CUAVs are unavailable or destroyed. Therefore, the system is resilient to the single-point-of-failure of key management. While a higher M value offers better resilience to such failure, it comes at the cost of performance. (ii) Regarding the network, the single-point-of-failure event is resolved by leveraging the blockchain network. Multiple copies of the ledger are maintained across the blockchain network to eliminate the possibility of single-point-of-failure.

5) *WUAV Re-Enforcement*: The proposed system offers a novel feature that allows GCS to re-enforce a remote cluster with new WUAVs. Consider that GCS had no access to the latest information on the cluster key or blockchain data. GCS decides to re-enforce a cluster using WUAV_r whose key pair is (WSK_r, WPK_r) . A payload is created where GCS uses TSM to sign and certify the public key WPK_r . On the other hand, WUAV_r generates the SSM of the payload. GCS then equips WUAV_r with the original LCU and the newly constructed payload. When WUAV_r reaches the remote destination, it selects CUAV[s] from LCU to process its re-enforcement request. It sends a request as shown in (14).

$$\begin{aligned} &\eta 7, \text{sig}\{ \text{hsh}\{ -, WPK_r, \eta 7 \} \}_{WSK_r}, \\ &GAST, WPK_r, \eta 8, \text{sig}\{ \text{hsh}\{ GAST, \\ &WPK_r, \eta 8 \} \}_{GSK}\}_{CPK[s]} \end{aligned} \quad (14)$$

The payload is encrypted using the public key $CPK[s]$ as the latest cluster key is unavailable to WUAV_r. CUAV[s] receives the request and decrypts the payload using its private key. Then SSM is used to validate the public key of WPK_r . TSM is used to validate and confirm that the GCS had dispatched WUAV_r. CUAV[s] then creates a new crypto-asset of $WAST\#r$ and associates it with WPK_r , using (3). Note that a single CUAV can complete the re-enforcement procedure, and multiple consents are not required. Consequently, CUAV[s] encrypts the up-to-date LWU and cluster key CK_1 using WPK_r and sends it to the WUAV_r. Further, CUAV[s] broadcasts WPK_r to other WUAVs of the cluster, as shown in (15). Note that the SSM of the new payload is constructed using TSM from (14).

$$\begin{aligned} &CUAV[s] \rightarrow WUAV[x] : \{ \text{enc}\{ GAST, WPK_r, \eta 8, \\ &\text{sig}\{ \text{hsh}\{ GAST, WPK_r, \eta 8 \} \}_{GSK}, CAST\#s, \\ &WAST\#r, \eta 9, \text{sig}\{ \text{hsh}\{ CAST\#s, WAST\#r, \eta 9 \} \\ &\}_{CSK[s]} \}_{CK_1} \} \end{aligned} \quad (15)$$

When WUAV receives the broadcast, it validates the (i) SSM to confirm that GCS dispatched the WUAV_r and (ii) TSM to ascertain that a member CUAV had approved the re-enforcement. Then $(WAST\#r, WPK_r)$ is added as the $(T - N)^{th}$ entry of the LWU. Broadcasting to other CUAVs is unnecessary as key information of WUAV_r is retrievable directly from their blockchain ledger. Note that the proposed system does not support re-enforcement with a new CUAV.

6) *Sub-Clustering and Re-Merging*: Sub-clustering is the division of a large UAV cluster into smaller but fully independent and functional sub-clusters. For a cluster with $M:N:T$ configuration, the maximum number of derivable sub-clusters is N/M . Each sub-cluster would have at least M CUAVs to carry out key management operations independently. Sub-clustering provides a flexible and scalable network with a wider coverage area. However, communication between sub-clusters is maintained via their respective CUAVs to synchronize their blockchain ledger. Re-merging of sub-clusters requires no additional permission or authentication. LCU, LWU, and blockchain are available to account for every UAV during the re-merger.

7) *Inter-Cluster Migration*: The proposed system allows WUAV to migrate from a cluster MSA to another cluster MSA' if they share the same blockchain network. Migration is performed simply by changing ownership of the crypto-asset of a WUAV from MSA to MSA'. For instance, if WUAV[m] wants to migrate from cluster MSA to cluster MSA', it sends a migration request to CUAV[0], similar to (7). CUAV[0] accepts the request and creates a multisig transaction based on (8). Subsequent steps to complete the migration process are similar to those of (9), (10), (11), and (12). Consequently, ownership of the crypto-asset $WAST\#m$ is changed to MSA'. As a result, only the cluster MSA' can now perform key management operations on the crypto-asset $WAST\#m$.

$$WUAV_r \rightarrow CUAV[s] : \{ \text{enc}\{ -, WPK_r,$$

8) *Optimization of Cluster Communication*: Since UAVs operate within a highly dynamic network, their network topology changes constantly and may delay the collection process of consent signatures. Therefore, an optimized network topology must be maintained to prevent such delay. For this purpose, necessary conditions include (i) maintenance of a direct CUAV-CUAV link to ensure efficient signature collection with minimum delay, and no involvement of intermediate WUAVs (ii) maintenance of a direct CUAV-WUAV link to ensure that WUAV communicates key management requests directly to a CUAV, and no involvement of other intermediate WUAVs. The sub-clustering discussed in Section VI-E6 is a useful feature that can facilitate assembly for an optimized network.

Moreover, delay due to multisignatures can be addressed based on (i) *communication latency*: $M=1$ causes no delay as a single CUAV can independently complete key management operation. $M>1$ causes delay due to additional signatures requirement for approval. To resolve this, the signature collection can be improved by optimizing network topology as discussed in Section VI-E8, (ii) *computational latency*: multisignature scheme should use efficient cryptographic primitives that ensure both security and performance. For the same security level, ECC-based cryptographic primitives use smaller key sizes. This makes ECC more efficient and suitable for resource-constrained devices. In addition, ECDSA is a relatively efficient digital signature scheme that generates compact signatures.

9) *Operation Under Hostile Environment*: CUAVs form the backbone of the proposed key management system and should be protected against different attacks. When the UAV cluster is deployed to a hostile environment, techniques such as (i) *sub-clustering* discussed in Section VI-E6 may be adopted to divide the main cluster into multiple sub-clusters before its deployment to the front line. At least one sub-cluster may be maintained as a backup so that any UAVs surviving the front line may re-merge with it, (ii) *re-enforcement* discussed in Section VI-E5 may be utilized to replete the cluster with additional WUAVs whenever needed, and (iii) *optimization of cluster communication* discussed in Section VI-E8 may be considered to maintain intra-cluster connectivity during any operational scenario.

10) *Crypto-Asset Recovery & Dispute Resolution*: In a $M:N:T$ cluster configuration, ownership of a crypto-asset and its associated key is distributed among N CUAVs. The recovery of such crypto-assets inherently relies on the cooperation of at least M CUAVs. As long as M CUAVs are active, the crypto-asset can be retrieved from the blockchain to carry out a key management operation. Similarly, resolutions of crypto-asset disputes rely on consensus among the M CUAVs to prevent any deadlock during key management operations.

VII. SECURITY ANALYSIS METHODOLOGY

Security analysis is performed on the (i) proposed key management system, (ii) proposed intra-cluster communication protocol, and (iii) security threats to the UAV cluster.

A. Security Analysis of the Proposed Key Management System

The proposed key management system is analyzed based on the National Institute of Standards and Technology (NIST)

guidelines for key management [33]. This guideline establishes a minimal level of security for key management systems. It identifies four different scopes that evaluate: (i) the support for protection requirement of key information. Only the key type of *public authentication key* is considered since the proposed system uses the public key as an identity, (ii) the support for recommended key states and transitions. This is important as different key states have distinct restrictions on key utilization, and (iii) the support for key management phases and functions. The proposed system is required to support recommended phases and their functions, and (iv) whether a focus is given on additional design and operational concerns raised due to the real-time nature of key management operations.

B. Security Analysis of the Intra-Cluster Communication Protocol

The proposed key management involves intra-cluster communication and is secured using the following techniques:

- A shared cluster key encrypts and decrypts communication between UAVs. Moreover, the cluster key is updated regularly to mitigate potential key leakage.
- Communication uses the two signature modules of SSM and TSM. SSM and TSM are used to authenticate the source and the intermediate participant, respectively.
- Nonce values are used to secure communication by preventing it from replay attacks.

The proposed system pre-configures a trusted source using (i) blockchain for CUAVs and (ii) LCU/LWU for WUAVs. The communication protocol uses such trust sources to perform one-way authentication. Therefore, the protocol is evaluated using the notion of *weak authentication*. Automated Validation of Internet Security Protocols and Applications (AVISPA) [34] tool is utilized to perform the analysis. Two analyzer models are also considered, an On-the-fly Model Checker (OFMC) and a Constraint-Logic-based Attack Searcher (CL-AtSe).

C. Security Analysis of Threats to the UAV Cluster

The safety of the UAV cluster reveals the effectiveness of the proposed key management system. For this purpose, the STRIDE Threat Model [35] is considered to identify potential security threats to the UAV cluster. These security threats are assessed using the Common Vulnerability Scoring System (CVSS) [36] and rated per CVSS v3.1. Individual UAVs (and GCS) are considered the vulnerable component, and the UAV cluster is the impacted component. In addition, the risk level of these security threats is also evaluated based on the NIST's guideline for conducting risk assessments [37]. Moreover, risk assessment on other advanced security attacks [38] targeting the different Open Systems Interconnection (OSI) layers is performed for the UAV cluster.

VIII. PERFORMANCE ANALYSIS METHODOLOGY

Performance analysis of the proposed system is conducted by implementing it in the UAV cluster. The following steps are considered for this purpose.

TABLE II
OMNET++ CONFIGURATIONS

Parameters	Value
Max. communication range	1000m
Bitrate	24Mbps
IEEE 802.11 mgmt. module	Ieee80211MgmtAdhoc
Transmission protocol	UDP
Public Key Cryptography	ECC (secp256r1)
Symmetric Cryptography	AES-128-CTR
Digital signature scheme	ECDSA (secp256r1)
Request interval	exponential(0.1s)
sim-time-limit	60s
Mode	Express

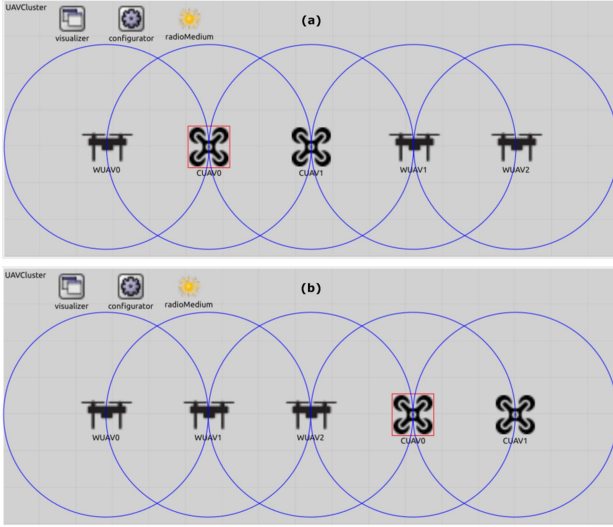


Fig. 4. Topologies for (a) best-case, and (b) worst-case.

A. Platform Configuration

OMNET++ [39] is used to implement and conduct the performance analysis of the proposed key management system. The different settings of the simulation platform are given in Table II. OMNET++ is installed in a virtual 64-bit Ubuntu-20.04.5 LTS with 2 GB RAM running in an Oracle VM Virtual-Box. The VirtualBox is installed in a 64-bit Windows 11 PC with an Intel Core i3 CPU (3.60 GHz) and 8.00 GB RAM. Crypto++ [40] is used as the library to perform cryptographic-related calculations.

B. Factors Considered for Performance Analysis

Different factors affecting the performance of the proposed system are identified. These factors include the (i) parameter M whose values are selected as $M = \{2, 3, 4, 5, 6\}$, (ii) parameter T whose values are selected as $T = \{25, 50, 75, 100\}$, (iii) case scenarios of network topology as {Best-case, Worst-case}. These scenarios are defined as follows:

- **Best-case:** Request passes through the shortest communication path to collect M consent signatures. This path consists only of CUAVs positioned consecutively, one after another. For example, a best-case topology for the 2:2:5 configuration is given in Fig. 4(a).
- **Worst-case:** Request passes through the longest communication path to collect M consent signatures. This path

comprises every WUAV and M CUAVs in the cluster. The CUAV[M-1] is at the end of the path. For example, a worst-case topology for the 2:2:5 configuration is given in Fig. 4(b).

In both cases, WUAV[0] sends the request, and CUAV[M-1] submits it to the blockchain. To ensure that the experimentation is reproducible, it is assumed that the UAVs are at the maximum communicable distance from one another. *LinearMobility* is selected as the Mobility model with a speed of 10 mbps, so the topology remains unchanged during the entire communication time. In addition, the response follows the same path as a request but in the reverse order.

The performance analysis also considers routing protocols of reactive and proactive types. Hence, the Ad Hoc On-Demand Distance Vector (AODV) and Destination-Sequenced Distance-Vector (DSDV) are used for this purpose.

C. Domains of the Performance Analysis

The performance analysis focuses on the three domains of (i) UAV cluster network, (ii) blockchain network, and (iii) energy consumption. It is conducted by evaluating the system during the key update operation.

1) *UAV Cluster Network Performance Analysis:* Two network performance metrics are considered, and they are defined as follows:

- **Average Response Delay:** This is the average waiting time for an update request to receive its corresponding response. For topologies given in Fig. 4, it is the average waiting time for an update request from WUAV[0] to receive its corresponding response from CUAV[M-1].
- **Total Response Loss:** This is the total number of update requests sent that never received their corresponding responses. For topologies given in Fig. 4, it is the total number of update requests sent by WUAV[0] but never received their corresponding response from CUAV[M-1].

In addition, *network bandwidth* is also studied by analyzing the average rate at which data is uploaded and downloaded through the network.

2) *Blockchain Network Performance Analysis:* This analysis focuses on monitoring the number of transactions submitted by the system versus the (i) number of blocks created by the blockchain network, and (ii) number of new entries added to the unspent transaction output (UTXO) set. While the submitted transactions indicate the throughput of the proposed key management system, the blocks created indicate the corresponding capability of the blockchain network to process transactions. On the other hand, a change in the UTXO set's entries reveals the memory usage of the blockchain server. This analysis aims to uncover any possible scalability bottlenecks in the blockchain network.

3) *Energy Consumption Analysis:* UAVs are energy-constrained devices whose power consumption directly affects the network's performance. This analysis examines the total energy consumption of UAVs in the cluster. Based on OMNET++, *IdealEpEnergyStorage* and *StateBasedEpEnergyConsumer*

TABLE III
NIST: PROTECTION REQUIREMENTS FOR KEY INFORMATION (OF PUBLIC AUTHENTICATION KEY)

		Supported?
Security Service	<i>Identity authentication</i>	✓
	<i>Integrity authentication</i>	✓
Security Protection	<i>Integrity</i>	✓
	<i>Availability</i>	✓
Association Protection	<i>Usage or application</i>	✓
	<i>Key pair owner</i>	×
	<i>Authenticated data</i>	✓
	<i>Private authentication key</i>	✓
	<i>Domain parameters</i>	×
Assurances Required	<i>Validity</i>	✓
Period of Protection	<i>From generation until no protected data needs to be authenticated</i>	✓

TABLE IV
NIST: KEY STATES AND TRANSITIONS

	Supported?
<i>Pre-Activation</i>	×
<i>Active</i>	✓
<i>Deactivated</i>	✓
<i>Compromised</i>	✓
<i>Suspended</i>	×
<i>Destroyed</i>	×

TABLE V
NIST: KEY-MANAGEMENT PHASES AND FUNCTIONS

		Supported?
Pre-operational	<i>Entity Registration</i>	✓
	<i>System Initialization</i>	✓
	<i>Initialization</i>	✓
	<i>Keying-Material Installation</i>	×
	<i>Key Establishment</i>	✓
	<i>Key Registration</i>	✓
Operational	<i>Normal Operational Storage</i>	✓
	<i>Continuity of Operations</i>	×
	<i>Key Change</i>	✓
	<i>Key Derivation Methods</i>	×
Post-operational	<i>Key Archive and Key Recovery</i>	✓
	<i>Entity De-registration</i>	✓
	<i>Key De-registration</i>	✓
	<i>Key Destruction</i>	×
	<i>Key Revocation</i>	✓
Destroyed	<i>Destroyed</i>	×

are chosen as the energy storage and consumption models, respectively.

IX. RESULTS AND DISCUSSION

The following sections highlight and discuss the results of security and performance analysis.

A. On Security Analysis

Firstly, the proposed key management system is analyzed based on NIST's guidelines for key management. Table III shows the analysis studied on the recommended properties for *public authentication key*. It reveals that most of the NIST's recommended properties are satisfied. However, Table IV discloses that the proposed system did not support all the recommended key states and transitions. Table V shows that most phases of the

TABLE VI
NIST: ADDITIONAL CONSIDERATIONS

	Addressed?
<i>Access Control & Identity Authentication</i>	✓
<i>Inventory Management</i>	✓
<i>Accountability</i>	✓
<i>Audit</i>	×
<i>Key-Management System Survivability</i>	✓

OFMC	CL-AtSe
% OFMC % Version of 2006/02/13 SUMMARY SAFE DETAILS BOUNDED_NUMBER_OF_SESSIONS PROTOCOL /home/span/span/testsuite/results/Sig2ModuleHLPsLif GOAL as_specified BACKEND OFMC COMMENTS STATISTICS parseTime: 0.00s searchTime: 0.00s visitedNodes: 7 nodes depth: 5 plies	SUMMARY SAFE DETAILS BOUNDED_NUMBER_OF_SESSIONS TYPED_MODEL PROTOCOL /home/span/span/testsuite/results/Sig2ModuleHLPsLif GOAL As Specified BACKEND CL-AtSe STATISTICS Analysed : 16 states Reachable : 7 states Translation: 0.00 seconds Computation: 0.00 seconds

Fig. 5. Communication protocol analyzed using AVISPA.

key lifecycle are implemented, except for the Destroyed phase. The recommended functions associated with these phases are also mostly supported. Table VI shows that the proposed system addressed most of the additional concerns about the real-time nature of key management operations.

Secondly, an AVISPA tool evaluates the proposed communication protocol that encrypts data and authenticates UAVs in the cluster. As shown in Fig. 5, it is observed that both the OFMC and CL-AtSe analyzer models confirmed the protocol as *SAFE*. Therefore, the proposed protocol can secure intra-cluster communication during threshold key management.

Thirdly, Table VII summarizes the security threats to the UAV cluster network, their CVSS scores, and corresponding mitigation techniques. The analysis reveals two critical security threats: (i) a compromised CUAV can generate a non-scheduled cluster key to hijack the whole network. Such a cluster key can compromise the confidentiality and privacy of cluster data and (ii) leakage of cluster information during the migration process. Denial of service attacks are another security concern that can disrupt normal key management services. Such attacks can also make the UAV cluster inaccessible and may shut it down completely. In addition, risk assessment corroborates that cluster key disclosure poses a very high risk to the network. It also reiterates the security risk due to a compromised CUAV that generates a non-scheduled cluster key. However, Table VII shows that the UAV cluster mitigates all these threats using the proposed threshold key management system. Furthermore, Table VIII reveals that the risk of attacks that target the transport, network, and data link layers is low as it requires the direct participation of a legitimate cluster member. However, the risk to the physical layer is higher as the UAV cluster network is wireless and open to attacks.

Moreover, the proposed system extensively uses ECC cryptosystems to generate public keys and digital signatures. The reason is that the ECC algorithm provides a similar security

TABLE VII
SECURITY ANALYSIS & RISK ASSESSMENT BASED ON STRIDE THREAT MODEL, CVSS

	Security Threats	CVSS Base Score	Risk Level	Countermeasures
Spoofing Identity	Non-member UAV impersonates WUAV/CUAV	7.6	Low	Identify using LCU/LWU/Blockchain
	WUAV impersonates CUAV	7.6	Low	Use LCU to identify CUAV
	WUAV impersonates other WUAVs	5.6	Low	Authenticate based on LWU
	Attacker impersonates GCS	7.9	High	Authenticate using the key info. of GAST
Tampering	UAV tampers LCU/LWU	5.2	Low	Verify from blockchain ledger
	CUAV tampers Blockchain	6.6	Moderate	Required 51% control of blockchain network
	CUAV/WUAV tampers payload	7.9	Low	Use signature modules of SSM and TSM
Repudiation	WUAV denies sending a request	5.0	Very Low	Request source verified using SSM & LWU
	CUAV denies giving consent signature	5.9	Very Low	Consent verifiable using TSM & LCU
	CUAV denies generating a cluster key	7.7	Very Low	Generator verifiable using SSM & LCU
	GCS denies re-enforcing a cluster	7.7	Very Low	GCS verifiable using the GAST crypto-asset
Information Disclosure	Cluster key discloses to non-member UAV	7.9	Very High	Cluster key is updated regularly
	Blockchain credentials disclosed	4.7	Very Low	Blockchain credentials are encrypted
	Non-member UAV eavesdropping	3.3	Moderate	Payload encrypted using cluster key
	Cluster info. leaks due to migration	9.6	Very High	Update cluster key after every migration
Denial of Service	Due to request flooding by WUAV/CUAV	8.6	Low	Accept & forward only verified payload
	Due to unavailability of CUAV	8.6	Low	Deployed multiple CUAVs
	Due to single-point-of-failure issue	8.6	Low	Deployed multiple CUAVs
	Due to sub-clustering	8.6	Low	Maintain M CUAVs in each N/M sub-cluster
Elevation of Privilege	WUAV signs consent signature	6.6	Low	Authenticate any signing UAV using LCU
	WUAV generates cluster key	7.2	Low	Authenticate generator CUAV using LCU
	Non-selected CUAV generates cluster key	9.9	High	Use round-robin scheduling/rebroadcast techniques

CVSS Score's Severity Rating Scale [None: 0.0, Low: 0.1 - 3.9, Medium: 4.0 - 6.9, High: 7.0 - 8.9, Critical: 9.0 - 10.0]

TABLE VIII
RISK ASSESSMENT OF ADVANCED SECURITY ATTACKS

Target of Attack	Attack Name	Risk Level
Transport Layer	SYN flood	Very Low
	UDP flood	Low
Network Layer	Black hole	Low
	Gray hole	Low
	Rushing	Low
	Wormhole	Low
Data Link Layer	Collision	Low
	De-authentication	Low
	Low link quality & high latency	Low
Physical Layer	Eavesdropping	High
	GPS Spoofing	Moderate
	Jamming	High

TABLE IX
SUMMARY OF THE PERFORMANCE ANALYSIS (KEY UPDATE OPERATION)

(in Average)	AODV		DSDV	
	Best	Worst	Best	Worst
Response delay (ms)	0.0061	2.5043	0.0060	0.0767
Response loss	0.2	278.25	76.35	401.75
Upload (bits/sec)	8991.75	46025.40	8483.42	27248.97
Download (bits/sec)	8991.38	45869.05	8321.93	26907.11
Transactions submit/sec	9.92	8.62	9.15	3.86
Blocks create/sec	0.31	3.98	1.28	2.20
UTXO entries/sec	19.84	17.23	18.29	7.73
Energy consumed (J)	0.01	0.15	0.13	0.14

level as other algorithms but with a smaller key size. However, the proposed system is not designed to be tightly coupled with ECC algorithms. Instead, it can be integrated with other cryptosystems and potentially even those designed for post-quantum security. Hence, the vulnerability of ECC algorithms to future advanced quantum computing is not a major concern to the proposed key management and the UAV cluster.

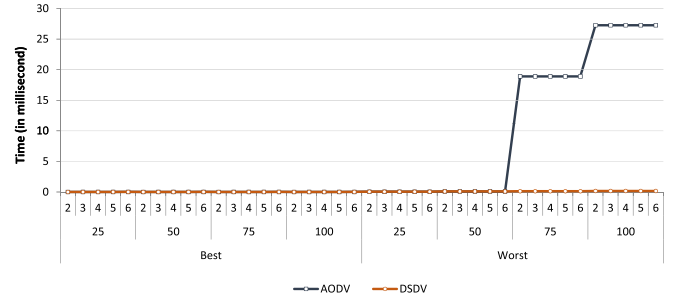


Fig. 6. Comparing average response delay.

B. On Performance Analysis

Performance analysis of the UAV cluster network is conducted based on the three factors of (i) best-case/worst-case scenarios, (ii) T parameter, and (iii) M parameter. The results of the performance analysis are discussed in the following sections.

1) *Network Performance*: Fig. 6 indicates that the average response delay of AODV and DSDV is consistent during the best-case scenario. However, the response delay for AODV becomes slower during the worst-case scenario as T value increases. Moreover, Fig. 7 indicates that during the best-case scenario, DSDV suffers response loss while AODV does not. However, both suffer from high response loss during the worst-case scenario due to large cluster size. Fig. 8 shows that the data transfer rate across the network is minimal during the best-case scenario. During the worst-case scenario, AODV consumed more network bandwidth due to higher upload/download of data.

2) *Blockchain Performance*: Fig. 9 compares transactions submitted against the corresponding (i) blocks generated, and (ii) UTXO Entries added. During the best-case scenario, AODV and DSDV exhibit similar performance patterns when generating transactions/blocks/UTXO entries. During the worst-case

TABLE X
COMPARISON WITH EXISTING KEY MANAGEMENT SYSTEMS FOR UAV NETWORKS

Key Management	Attribute	[20]	[21]	[22]	[23]	[24]	Proposed
UAV Network	Configurable	×	×	×	×	×	✓
	Collaborative/Threshold	×	×	×	✓	×	✓
	Autonomous	✓	×	✓	×	×	✓
	Re-enforcement	×	×	×	×	×	✓
Issue Addressed	Sub-clustering	×	×	×	×	×	✓
	Migration	✓	✓	×	✓	×	✓
	Single-point-of-failure	×	✓	×	✓	×	✓
	Compromised CA problem	×	×	×	✓	×	✓
Security Analysis (Tool/Model/etc.)	Key Management	×	×	×	×	×	NIST
	Communication	×	×	×	×	ProVerif	AVISPA
	UAV cluster network	Custom	×	×	Custom	Custom theorem	STRIDE
Performance Analysis	Delay/Latency	×	E2E 0.0226 bps	×	×	×	0.0767ms (avg)
	Loss	×	PKR 0.01ps	×	×	×	401.75 (avg)
	Energy Requirement	110J/Ctr. Head (max)	×	×	×	×	0.14 J (avg)
Blockchain Network	Transaction/Block rate	0.03tps (max)	8tps (max)	×	50tps	×	3.86tps (avg)
	UTXO entries	×	×	×	×	×	7.73/s (avg)

Due to the non-compatibility of experimentation/simulation, the performance metrics given are for reference purposes only.

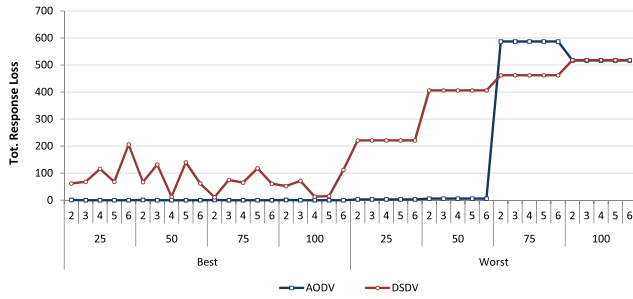


Fig. 7. Comparing total response loss.

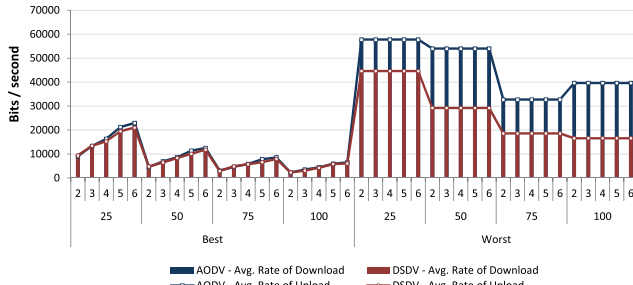


Fig. 8. Comparing network bandwidth usage.

scenario, DSDV generated fewer transactions resulting in fewer blocks/UTXO entries in the blockchain network. Therefore, DSDV provides better blockchain performance than AODV, as it requires fewer computational and memory resources.

3) *Energy Consumption*: Fig. 10 reveals that AODV consumes less energy than DSDV during the best-case scenario. During the worst-case scenario, AODV and DSDV have similar energy consumption patterns.

A summary of the performance analysis is given in Table IX. Both AODV and DSDV exhibit mostly similar performance metrics during the best-case scenario. However, DSDV delivers higher performance in worst-case scenarios, especially regarding response delay, bandwidth usage, and blockchain efficiency. Further, a comparison of the proposed system with the existing key management systems is also performed and shown in Table X.

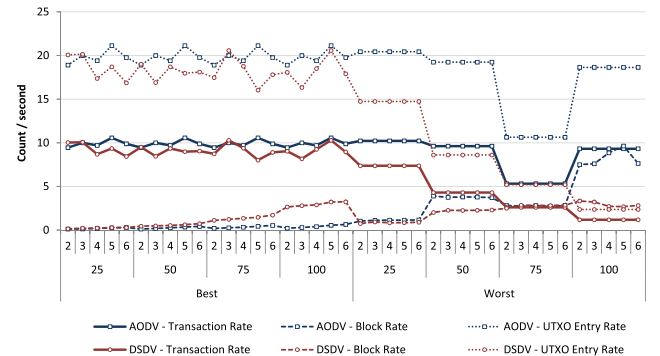


Fig. 9. Transactions submitted vs. Blocks created vs. UTXO Entries added.

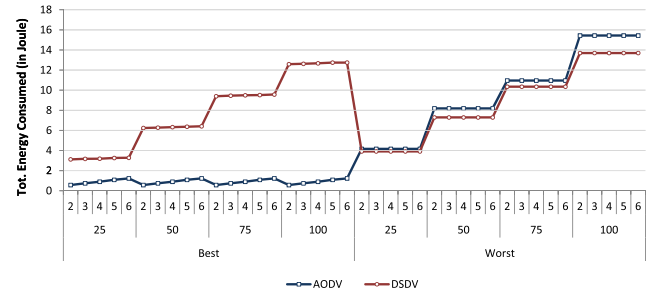


Fig. 10. Comparing Energy consumption.

X. FUTURE SCOPE OF WORK

Investigation of the proposed key management system and the UAV cluster infers that there is scope for further research. The UAV cluster has assumed a predefined path to collect consent signatures. A practical mechanism may be devised to optimize the signatures collection process. Such a mechanism should aim toward building an optimal routing path to forward requests, thus causing minimum delay. To secure the UAV cluster, the number of compromised CUAVs should not exceed the security threshold value of M . A checking mechanism should be adopted to prevent compromised CUAVs from teaming up and disrupting the system. Further, the effect of *re-enforcement*, *sub-clustering*, *re-merging*, and *inter-cluster migration* techniques on the cluster

performance require further investigation. The need to address attacks on UAV clusters, such as signal jamming, GPS spoofing, and other forms of electronic warfare, is critical. Moreover, the cluster should be evaluated using routing protocols like Optimized Link State Routing (OLSR), Dynamic Source Routing (DSR), Greedy Perimeter Stateless Routing (GPSR), etc., and under real-life scenarios using dynamic topology and mobility. Support to other blockchain platforms like Ethereum, Hyperledger, and Directed Acyclic Graph (DAG) ledger should be extended. DAG is an emerging distributed ledger technology that allows simultaneous transaction processing. It is capable of reducing confirmation time and improving network throughput. Therefore, adopting the DAG protocol could enhance the performance and scalability of the proposed system and the UAV cluster.

XI. CONCLUSION

An autonomous UAV cluster was secured using the blockchain-based threshold key management system. Key management operations were carried out independently without intervention from any ground-based stations. The proposed system treated key information of UAVs as blockchain-based crypto-assets. Crypto-assets enabled the creation, updating, transfer, and revocation of key information within the blockchain network. Multisignature distributed key management authority among multiple UAVs to resolve compromised CA problems and single-point-of-failure events. The system was designed to be configurable, allowing it to adapt to various security and performance requirements for UAV applications.

The proposed key management system was verified to comply with most of the NIST's guidelines. The proposed intra-cluster communication protocol was also verified using AVISPA and found to be secure. Security threats to the UAV cluster were identified using the STRIDE Threat model and their risk assessment was performed. These threats were then mitigated using the proposed key management system. Risk assessment found that attacks could occur at the physical layer due to the wireless nature of the network. Performance analysis revealed that AODV and DSDV performed consistently during the best-case scenario, but DSDV performance exceeded during the worst-case scenario. It noted that cluster size T had a visible effect on the performance of the routing protocols and proposed system. The blockchain analysis showed that AODV generated more transactions/blocks, increasing the consumption of computational resources, bandwidth, and storage. Finally, the energy analysis showed that AODV and DSDV had similar energy consumption patterns, where consumption increases with increasing M and T values. However, DSDV consumed more energy during the best-case scenario.

Future work should focus on the security and performance of the proposed key management system. This included designing an efficient consent signature collection mechanism, identifying an optimized security threshold, and investigating performance issues related to re-enforcement, sub-clustering, re-merging, and inter-cluster migration.

REFERENCES

- [1] G. K. Pandey, D. S. Gurjar, H. H. Nguyen, and S. Yadav, "Security threats and mitigation techniques in UAV communications: A comprehensive survey," *IEEE Access*, vol. 10, pp. 112858–112897, 2022.
- [2] A. Fotouhi et al., "Survey on UAV cellular communications: Practical aspects, standardization advancements, regulation, and security challenges," *IEEE Commun. Surveys Tut.*, vol. 21, no. 4, pp. 3417–3442, Fourth Quarter 2019.
- [3] Y. Mekdad et al., "A survey on security and privacy issues of UAVs," *Comput. Netw.*, vol. 224, 2023, Art. no. 109626.
- [4] D. P. Agrawal, H. Deng, and A. Mukherjee, "Threshold and Identity-Based Key Management and Authentication for Wireless ad Hoc Networks," US Patent 8,050,409, Nov. 1, 2011.
- [5] A. Diop, Y. Qi, and Q. Wang, "Efficient group key management using symmetric key and threshold cryptography for cluster based wireless sensor networks," *Int. J. Comput. Netw. Inf. Secur.*, vol. 6, no. 8, 2014, Art. no. 9.
- [6] Y. Zhang, J. Liu, Y. Wang, J. Han, H. Wang, and K. Wang, "Identity-based threshold key management for ad hoc networks," in *Proc. IEEE Pacific-Asia Workshop Comput. Intell. Ind. Appl.*, 2008, pp. 797–801.
- [7] A. Kumar, A. Aggarwal, and Charu, "Efficient hierarchical threshold symmetric group key management protocol for mobile ad hoc networks," in *Proc. 5th Int. Conf. Contemporary Comput.*, Noida, India, 2012, pp. 335–346.
- [8] Y. Tan, J. Wang, J. Liu, and N. Kato, "Blockchain-assisted distributed and lightweight authentication service for industrial unmanned aerial vehicles," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 16928–16940, Sep. 2022.
- [9] R. Chen, H.-W. Tseng, J.-L. Lien, and W. Liao, "Blockchain-empowered identity management with a dual identity model for UAVs in 5G networks," *IEEE Internet Things Mag.*, vol. 5, no. 2, pp. 69–73, Jun. 2022.
- [10] K. Gai, Y. Wu, L. Zhu, K.-K. R. Choo, and B. Xiao, "Blockchain-enabled trustworthy group communications in UAV networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4118–4130, Jul. 2021.
- [11] K. Yu et al., "A blockchain-based shamir's threshold cryptography scheme for data protection in industrial Internet of Things settings," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8154–8167, Jun. 2022.
- [12] T. Ankenbrand, D. Bieri, R. Cortivo, J. Hoehener, and T. Hardjono, "Proposal for a comprehensive (crypto) asset taxonomy," in *Proc. Crypto Valley Conf. Blockchain Technol.*, 2020, pp. 16–26.
- [13] P. Freni, E. Ferro, and R. Moncada, "Tokenization and blockchain tokens classification: A morphological framework," in *Proc. IEEE Symp. Comput. Commun.*, 2020, pp. 1–6.
- [14] J. Arcenegui et al., "Secure management of IoT devices based on blockchain non-fungible tokens and physical unclonable functions," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.*, 2020, pp. 24–40.
- [15] U. Khalil, O. A. Malik, O. W. Hong, and M. Uddin, "DSCoT: An NFT-based blockchain architecture for the authentication of IoT-enabled smart devices in smart cities," 2022, *arXiv:2211.04803*.
- [16] D. Zelenyanszki, Z. Hóu, K. Biswas, and V. Muthukumarasamy, "A privacy awareness framework for NFT avatars in the metaverse," in *Proc. Int. Conf. Comput. Netw. Commun.*, 2023, pp. 431–435.
- [17] B. Rajendran and A. K. Pandey, "PKI ecosystem for reliable smart contracts and NFT," in *Proc. IEEE Int. Conf. Public Key Infrastructure Appl.*, 2022, pp. 1–5.
- [18] C. Rong, J. Geng, and M. G. Jaatun, "Managing digital objects with decentralised identifiers based on NFT-like schema," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci.*, 2022, pp. 246–251.
- [19] S. Ghirmal, D. Mebrahtom, M. Aloqaily, M. Guizani, and M. Debbah, "Self-sovereign identity for trust and interoperability in the metaverse," in *Proc. IEEE Smartworld Ubiquitous Intell. Comput. Scalable Comput. Commun., Digit. Twin Privacy Comput. Metavers Auton. Trusted Veh.*, 2022, pp. 2468–2475.
- [20] Y. Tan, J. Liu, and N. Kato, "Blockchain-based key management for heterogeneous flying ad hoc network," *IEEE Trans. Ind. Informat.*, vol. 17, no. 11, pp. 7629–7638, Nov. 2021.
- [21] A. Yazdinejad, R. M. Parizi, A. Dehghantanha, H. Karimipour, G. Srivastava, and M. Aledhari, "Enabling drones in the Internet of Things with decentralized blockchain-based security," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6406–6415, Apr. 2021.
- [22] N. Jäger and A. Abmuth, "An approach for decentralized authentication in networks of UAVs," in *Proc. 12th Int. Conf. Cloud Comput.*, 2021, pp. 13–17.
- [23] C. Feng, B. Liu, Z. Guo, K. Yu, Z. Qin, and K.-K. R. Choo, "Blockchain-based cross-domain authentication for intelligent 5G-enabled Internet of Drones," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 6224–6238, Apr. 2022.

- [24] S. U. Jan and H. U. Khan, "Identity and aggregate signature-based authentication protocol for IoD deployment military drone," *IEEE Access*, vol. 9, no. 15, pp. 130247–130263, Apr. 2022.
- [25] F. Bao, R. H. Deng, and H. Zhu, "Variations of Diffie–Hellman problem," in *Information and Communications Security*, S. Qing, D. Gollmann, and J. Zhou, Eds., Berlin, Germany: Springer, 2003, pp. 301–312.
- [26] G. Heo, K. Chae, and I. Doh, "Hierarchical blockchain-based group and group key management scheme exploiting unmanned aerial vehicles for urban computing," *IEEE Access*, vol. 10, pp. 27990–28003, 2022.
- [27] H. Xie, J. Zheng, T. He, S. Wei, C. Shan, and C. Hu, "B-UAVM: A blockchain-supported secure multi-UAV task management scheme," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21240–21253, Dec. 2023.
- [28] K. Hidawi, "Dronexnft: An NFT-driven framework for secure autonomous UAV operations and flight data management," 2024, *arXiv:2409.06507*.
- [29] A. Yazdinejad, A. Dehghantanha, R. M. Parizi, G. Srivastava, and H. Karimipour, "Secure intelligent fuzzy blockchain framework: Effective threat detection in iot networks," *Comput. Ind.*, vol. 144, 2023, Art. no. 103801. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016636152200197X>
- [30] A. M. Antonopoulos, *Mastering Bitcoin: Unlocking Digital Cryptocurrencies*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2014.
- [31] C. S. Ltd, "Multichain json-rpc api commands," Aug. 20, 2023. [Online]. Available: <https://www.multichain.com/developers/json-rpc-api/>
- [32] B. Project, "Bitcoin core," Sep. 9, 2023. [Online]. Available: <https://bitcoin.org/en/bitcoin-core/>
- [33] E. Barker, "Recommendation for key management: Part 1–general," *NIST Special Publication 800-57 Part 1, Revision 5*, 2020, doi: [10.6028/NIST.SP.800-57pt1r5](https://doi.org/10.6028/NIST.SP.800-57pt1r5).
- [34] T. Team et al., "Avispa v1.1 user manual," Sep. 22, 2024. [Online]. Available: https://people.irisa.fr/Thomas.Genet/Crypt/AVISPA_manual.pdf
- [35] R. Khan, K. McLaughlin, D. Laverty, and S. Sezer, "Stride-based threat modeling for cyber-physical systems," in *Proc. IEEE PES Innov. Smart Grid Technol. Conf. Europe*, 2017, pp. 1–6.
- [36] P. Mell et al., "A complete guide to the common vulnerability scoring system version 2.0," *FIRST-Forum Incident Response Secur. Teams*, vol. 1, 2007, Art. no. 23.
- [37] R. S. Ross, "Guide for conducting risk assessments," *NIST Special Publication 800-30, Revision 1*, 2012, doi: [10.6028/NIST.SP.800-30r1](https://doi.org/10.6028/NIST.SP.800-30r1).
- [38] K.-Y. Tsao, T. Girdler, and V. G. Vassilakis, "A survey of cyber security threats and solutions for UAV communications and flying ad-hoc networks," *Ad Hoc Netw.*, vol. 133, 2022, Art. no. 102894. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1570870522000853>
- [39] O. Ltd, "Omnet discrete event simulator," Aug. 27, 2023. [Online]. Available: <https://omnetpp.org/>
- [40] C. Project, "Crypto 8.8 release," Sep. 22, 2024. [Online]. Available: https://github.com/weidai11/cryptopp/releases/tag/CRYPTOPP_8_8_0



Mebanjop Kharjana received the BE degree in computer engineering from Mumbai University, India, and the MTech degree in information technology from North-Eastern Hill University, Shillong, India. He is currently working toward the PhD degree with the Department of Information Technology, North-Eastern Hill University, Shillong, India. He is working as a system analyst with the North-Eastern Hill University, Shillong, India. His research interests include IoT security, information-centric networking, and blockchain technology.



Subhas Chandra Sahana received the PhD degree from the North-Eastern Hill University, Shillong, India, and the MTech degree from the Tezpur University, Assam, India. He is an assistant professor with the Department of Computer Science & Engineering, National Institute of Technology, Durgapur, West Bengal, India. His current research interests include IoT security and elliptic curve cryptography.



Goutam Saha received the BE degree in electrical engineering and the ME degree in electronics and telecommunication engineering from the Bengal Engineering College, Shibpur (then under the University of Calcutta), in 1984 and 1989, respectively, and the PhD degree from the Indian Institute of Technology, Kharagpur, in 1999. He also has postdoctoral research experience from the Ben Gurion University, Israel. He is currently a professor with the Department of Information Technology, North-Eastern Hill University, Shillong, India. His current research interests include computational biology, bioinformatics, systems biology, river networking, IoT, and Big Data.